МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: М. А. Седов
Группа: М8О-316БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы — построить трёхмерную модель правильного октаэдра средствами Vulkan и реализовать интерактивное управление сценой. Приложение написано на C++20; окно создаётся через GLFW, панель управления — через ImGui.

Был выполнен вариант 2 которому соответствует правильный октаэдр.

В дополнение к основной модели реализованы все шесть дополнительных заданий:

1. Переключение между перспективной и ортографической проекциями.

2. Перенос, вращение и масштабирование объекта по трём осям.

3. Движение по пространственной периодической траектории с паузой, настройкой скорости, радиуса и высоты.

4. Изменение цвета через панель управления.

5. Вычисление цвета вершин из локальных координат и умножение на выбранный цвет.

6. Отрисовка второго объекта с отдельным набором дескрипторов VkDescriptorSet.

Модель октаэдра

Модель содержит шесть вершин на координатных осях: (0, 1, 0), (1, 0, 0), (0, 0, 1), (−1, 0, 0), (0, 0, −1), (0, −1, 0). Индексный буфер задаёт восемь треугольников, по четыре у верхней и нижней вершин. Каждое из двенадцати рёбер имеет длину $\sqrt{2}$, а каждое ребро принадлежит двум граням. Таким образом, поверхность замкнута и все грани являются равносторонними треугольниками.

Метод решения

Преобразования и проекция

Вершины хранятся в локальной системе координат. Матрица модели M задаёт масштаб, вращение и перенос, матрица вида V переводит координаты в систему камеры, матрица проекции P формирует однородные координаты для последующей растеризации. В вершинном шейдере используется преобразование:

$$p_{clip} = P\,V\,M\,p, \quad M = T\,R_z\,R_y\,R_x\,S.$$

Матрицы умножаются на векторы-столбцы. Поэтому сначала применяется масштаб S, затем повороты вокруг осей X, Y, Z и в конце перенос T. В интерфейсе углы вводятся в градусах; перед вычислением матриц они переводятся в радианы.

Камера находится в точке (0, 0, 5). Для перспективной проекции используются ближняя плоскость 0,1 и дальняя плоскость 100. Удалённые части объекта отображаются меньшими. Ортографическая проекция сохраняет масштаб независимо от расстояния до камеры. Обе матрицы учитывают отношение сторон области сцены и диапазон глубины Vulkan от 0 до 1. Панель управления исключена из области отрисовки модели.

Буферы и наборы дескрипторов

Вершинный буфер хранит положение и цвет каждой вершины, индексный — 24 индекса для восьми треугольников. Геометрия общая для двух объектов. Для размещения буферов используется VMA. После записи в отображённую память вызывается vmaFlushAllocation, чтобы изменения были доступны GPU при работе с некогерентной памятью.

Данные сцены и объектов разделены. Набор set = 0 содержит SceneUniform с матрицами V и P размером 128 байт. Набор set = 1 содержит ObjectUniform с матрицей M и четырёхкомпонентным цветовым множителем размером 80 байт. Для каждого октаэдра выделены собственный uniform-буфер и собственный VkDescriptorSet. Перед очередным вызовом отрисовки переключается набор объекта; конвейер и буферы геометрии остаются общими. Выравнивание структур соответствует std140.

Шейдеры и проверка глубины

Локальные координаты вершин лежат в диапазоне от −1 до 1. Исходный RGB-цвет вычисляется по формуле c = (p + 1) / 2, после чего в вершинном шейдере покомпонентно умножается на цветовой множитель из ObjectUniform. Растеризатор интерполирует цвет внутри каждого треугольника; фрагментный шейдер выводит полученный цвет с непрозрачностью 1. Освещение и текстуры в данной работе не используются.

Графический конвейер собирает треугольники в режиме TRIANGLE_LIST. Для корректного перекрытия граней включены тест и запись глубины: фрагмент проходит, если его глубина меньше сохранённой. Буфер глубины очищается значением 1. Viewport и scissor задаются динамически. GLSL-шейдеры компилируются в SPIR-V утилитой glslc при сборке CMake.

Анимация и управление

К заданному в интерфейсе положению основного объекта прибавляется смещение по пространственной траектории:

$$a(t) = (r\ cos\ t;\ h\ sin\ 2t;\ 0{,}7r\ sin\ t).$$

Здесь r — радиус, h — высота, t — время анимации. Горизонтальная составляющая образует эллипс, а вертикальная совершает два колебания за один обход. Одновременно изменяются углы вращения. Время увеличивается на длительность кадра, умноженную на выбранную скорость. При паузе t остаётся постоянным, поэтому положение и ориентация не меняются. Сброс сцены восстанавливает исходные параметры и обнуляет время.

Порядок кадра и завершение работы

Перед записью uniform-буферов процессор ожидает fence предыдущего кадра. Затем обновляются параметры, записываются команды привязки конвейера, буферов и дескрипторов, выполняются индексные вызовы отрисовки обоих объектов и команды ImGui. После отправки command buffer изображение передаётся в очередь представления. При изменении размеров окна пересоздаётся swapchain, а при свёрнутом окне отрисовка пропускается.

При выходе приложение ожидает завершения работы устройства и освобождает конвейер, его layout, пул и layouts дескрипторов, буферы и выделенную память. Затем освобождаются ресурсы окна и Vulkan-контекста.

Результаты

На рисунках 1 и 2 показана одна сцена с остановленной анимацией. Изменена только проекция: в перспективном режиме основной октаэдр выглядит крупнее, а в ортографическом его размер определяется масштабом проекции. Оба объекта используют одну геометрию, но имеют разные матрицы модели и цветовые множители.

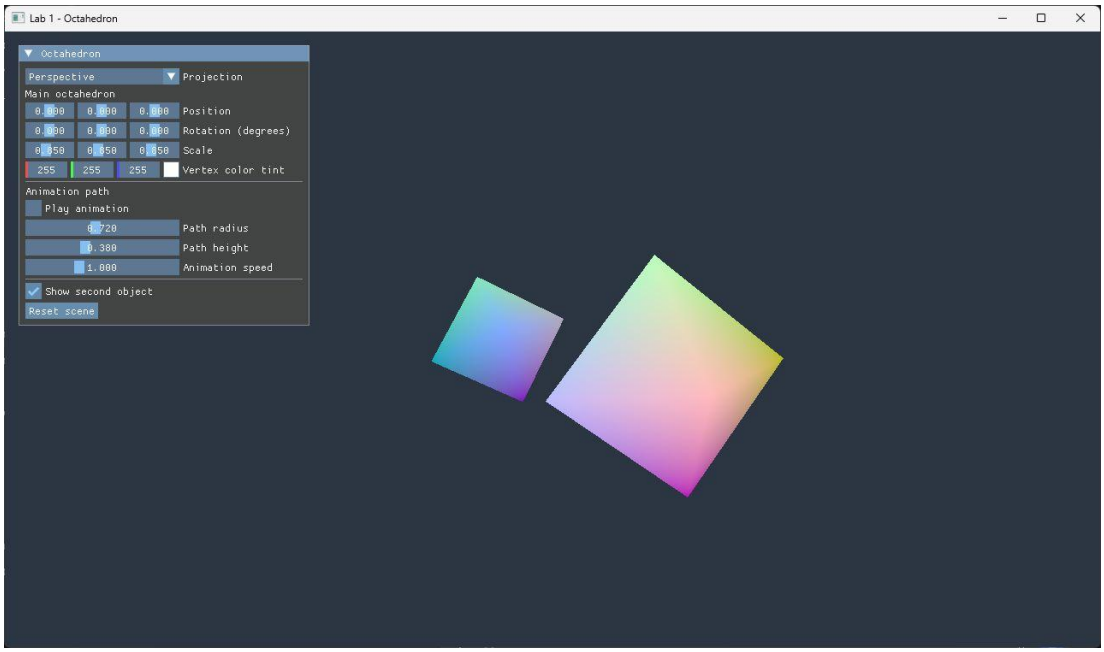


Рисунок 1 — Два октаэдра в перспективной проекции

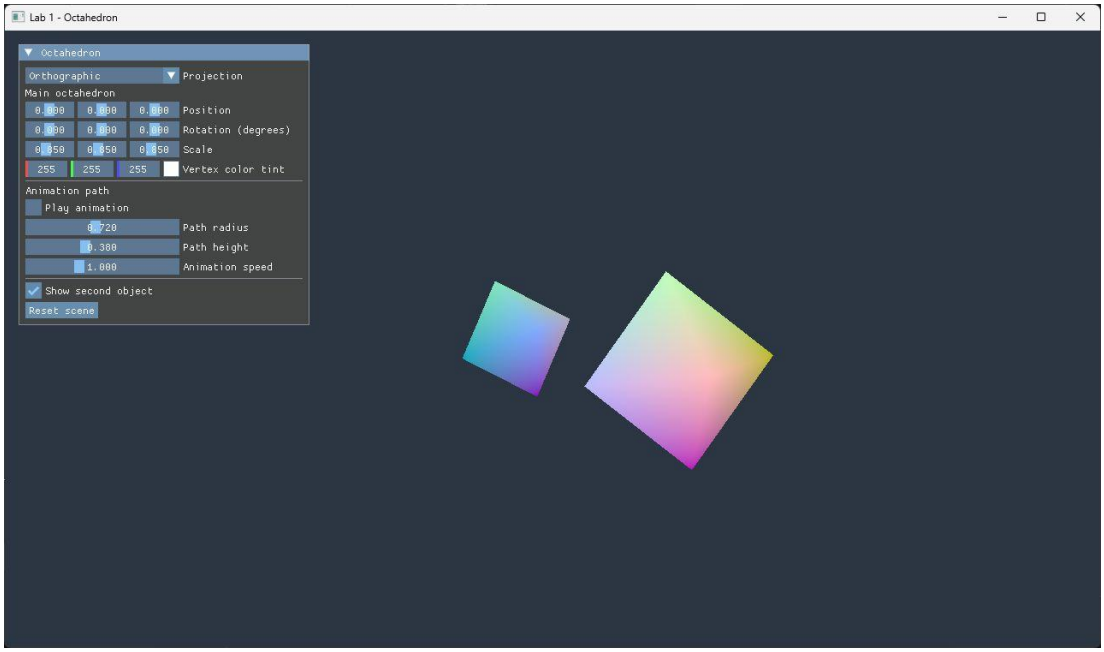


Рисунок 2 — Та же сцена в ортографической проекции

На рисунке 3 угол основного объекта вокруг оси X установлен в 180°, красная компонента цветового множителя уменьшена с 255 до 206. При этом второй объект сохраняет прежние параметры. Изменения применяются через отдельный набор дескрипторов основного объекта.

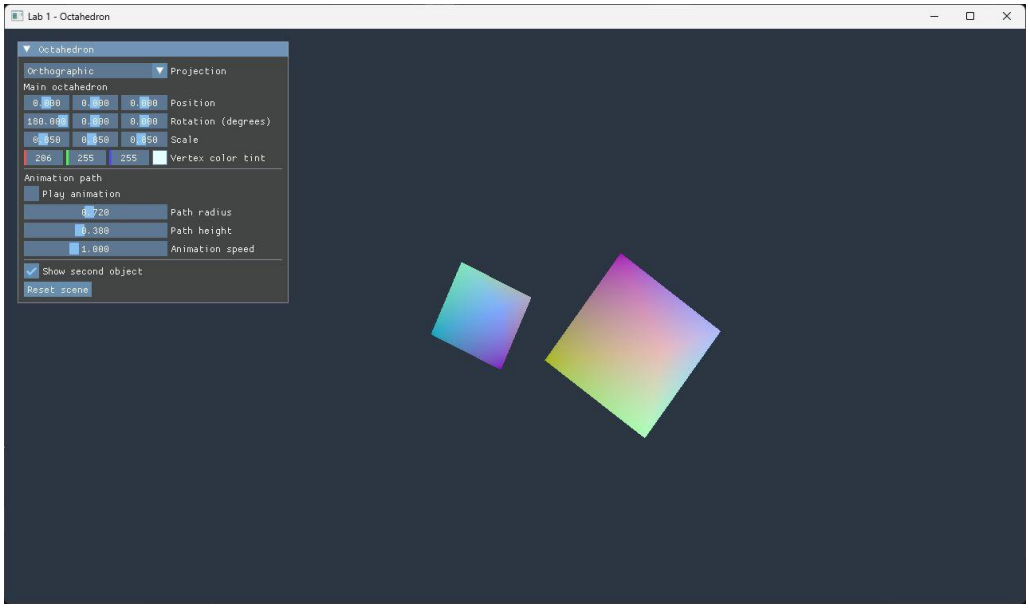


Рисунок 3 — Изменение вращения и цвета основного объекта

Проверка

Приложение собрано в конфигурации MSVC Debug и запущено на Intel UHD Graphics. Переключение проекций, остановка анимации, изменение вращения и цвета проверены в работающем окне. Снимки получены непосредственно из приложения.

При запуске и штатном завершении ошибок Vulkan с идентификаторами VUID в журнале не зарегистрировано.

Выводы

Реализована отрисовка правильного октаэдра и все шесть дополнительных заданий. Два объекта используют общую геометрию и конвейер, а независимые преобразования и цвета передаются через отдельные наборы дескрипторов. Проверены геометрия модели, матричные преобразования и работа интерфейса.